

Risma, Vincenzo Luis S. - DSS150P

1. Why should source profiling occur before implementation of ingestion?

Systemic architecture, including duplicate-key logic, watermark configurations, and validation parameters, necessitates prior empirical knowledge of the dataset. For example, developing API deduplication logic prior to profiling `api_events.json` could result in the erroneous assumption of `event_id` uniqueness. This would miss collisions like E0020 and E0055, leading to silent output corruption. Furthermore, initial profiling of `customers.csv` revealed that duplicate `customer_id` values represent genuine identity collisions rather than simple re-ingestion errors. Building ingestion pipelines before this discovery requires retroactively modifying core assumptions instead of designing around known conditions from the start.

2. What is the difference between source event time/updated_at and ingestion time?

The `updated_at` parameter represents the exact timestamp when an event occurred or was modified within the source system. This constitutes intrinsic business data. Conversely, `_ingested_at` denotes the operational timestamp when the pipeline extracted and recorded the data. These values can diverge significantly; an event with an `updated_at` of 2026-08-01 may reflect an `_ingested_at` of 2026-09-12 due to pipeline execution schedules. Watermark logic relies exclusively on `updated_at` to identify new source records, whereas `_ingested_at` serves strictly for operational auditing.

3. Why is event_id alone insufficient to decide which duplicate API record to keep in this exercise?

A duplicated `event_id` lacks the necessary context to determine the authoritative record. In the instances of E0020 and E0055, both copies share identical IDs but exhibit variant amount and `updated_at` values. Relying solely on `event_id` would necessitate arbitrary selection, potentially retaining stale, pre-update data based on pagination boundaries. The `updated_at` field systematically encodes chronological precedence, mandating a deduplication rule that retains the record with the maximum `updated_at` value.

4. Why must watermark state advance only after successful persistence?

The watermark functions as the exclusive state mechanism tracking durably captured records. Should the watermark advance prior to a successful raw write operation that subsequently fails, subsequent pipeline executions will query for records occurring after the uncommitted timestamp. This results in permanent data gaps without triggering systemic errors. Advancing the watermark strictly post-persistence ensures absolute parity between recorded progress and durably captured data.

5. What limitation does updated_after > watermark have when multiple source records can share exactly the same timestamp?

Utilizing a strict greater-than operator against a singular timestamp fails to differentiate between complete and partial sets of records sharing that exact timestamp. If a batch processing boundary intersects a group of concurrent records, a subset may be ingested while the remainder is permanently bypassed during the subsequent run, as the watermark has already exceeded that timestamp. While the evaluated API sorts by the compound key, the persisted watermark records only the timestamp component, introducing latent systemic risk.

6. How is duplicate prevention related to idempotency?

Idempotency defines an operational state where multiple executions of a process yield the exact same outcome as a single execution. Duplicate prevention serves as the structural mechanism enforcing this

property. Without it, repeated file ingestions would generate redundant data copies, and API pipelines would continuously append identical records. Employing SHA-256 manifest validation and event_id merge-and-deduplication logic ensures that the system evaluates existing logical states before executing write operations, effectively rendering repetitive executions as safe null operations.

7. Why should the raw area preserve source values instead of applying business transformations?

Applying immediate transformations permanently overwrites original source values unless parallel storage is maintained. For instance, prematurely resolving the C0090 identity collision during initial ingestion would permanently bake in an unverified assumption. Maintaining the raw layer as an exact, unmodified replica of the source data ensures that subsequent cleaning and transformation algorithms can be iterated or reversed without data loss.

8. How could querying a production OLTP source for profiling or extraction degrade the application?

Online Transaction Processing architectures are optimized for high-volume, localized transactions rather than comprehensive analytical scans. Executing unbounded profiling queries or full-table aggregations can monopolize system I/O and maintain database locks, directly inducing latency and contention for end-users. The constraints applied in this project dictate strictly bounded queries to circumvent this risk. While the current support_tickets dataset is negligible at 250 rows, applying unbounded extraction patterns at production scales guarantees significant degradation of live applications.

9. What would you change if the API had a rate limit of 60 requests per minute?

The existing pagination loop executes fetch_api_page continuously, which would rapidly violate a 60 requests-per-minute threshold. The architecture must be modified to incorporate programmatic delays between requests and explicit retry logic handling HTTP 429 errors. This involves parsing the Retry-After header and implementing exponential backoff instead of terminating the process. Additionally, configuring the per_page parameter to the API maximum of 50 would minimize total request volume, optimizing rate-limit budget consumption.

10. How would you extend this pipeline from a local raw area to PostgreSQL while preserving rerun safety?

Transitioning to a PostgreSQL architecture requires replicating rerun-safety guarantees within the relational database schema. The file-hash deduplication protocol translates to an INSERT ... ON CONFLICT DO NOTHING operation against a metadata manifest table. The event_id deduplication transforms into a conditional upsert utilizing INSERT ... ON CONFLICT DO UPDATE ... WHERE excluded.updated_at > existing.updated_at, maintaining identical logic at the database level. These operations, alongside watermark updates, must execute within singular, atomic transactions to guarantee structural consistency in the event of partial execution failures.